Atelier 7 : GitLab CI/CD + SonarQube Cloud

Rapport de travaux pratiques

Étudiante : Basma Elmaimouni

Projet : atelier-sonarqube

Technologies : HTML, CSS, JavaScript, GitLab CI/CD, SonarQube Cloud, GitLab Pages

1. Objectif

Mettre en place un pipeline GitLab CI/CD qui analyse automatiquement un mini-site web avec SonarQube Cloud, puis déploie le site sur GitLab Pages uniquement si le Quality Gate est respecté. Deux versions du projet sont utilisées :

- V1 : code correct.
- V2 : code volontairement vulnérable (faille XSS).

2. Présentation des outils

- GitLab CI/CD : automatise les étapes (analyse, déploiement) à chaque git push.
- SonarQube Cloud : analyse la qualité et la sécurité du code (bugs, vulnérabilités, Security Hotspots, Code Smells).
- Quality Gate : ensemble de conditions qui détermine si le code satisfait les exigences de qualité du projet.

3. Configuration

1. Création du projet public atelier-sonarqube sur GitLab et clonage en local avec git clone.
2. Connexion à SonarQube Cloud avec le compte GitLab, import de l'organisation (basmaelmaimouni6) puis création du projet basmaelmaimouni6_atelier-sonarqube.
3. Désactivation de l'Automatic Analysis, qui entre en conflit avec l'analyse lancée par le CI.
4. Création du fichier sonar-project.properties.
5. Création d'un jeton SonarQube et ajout des variables CI/CD SONAR_HOST_URL et SONAR_TOKEN dans GitLab.
6. Création du fichier .gitlab-ci.yml (jobs sonarqube-check et deploy-pages pour la V1).

Fichier sonar-project.properties :

```
sonar.projectKey=basmaelmaimouni6_atelier-sonarqube
sonar.organization=basmaelmaimouni6
sonar.projectName=Atelier CI CD SonarCloud

sonar.sources=.
sonar.sourceEncoding=UTF-8
sonar.exclusions=public/**

sonar.qualitygate.wait=true
```

3.1 Difficultés rencontrées

Deux problèmes ont été rencontrés lors du premier lancement du pipeline. Le premier pipeline a échoué sans exécuter de job, car GitLab demandait de vérifier le compte avant d'utiliser les pipelines :

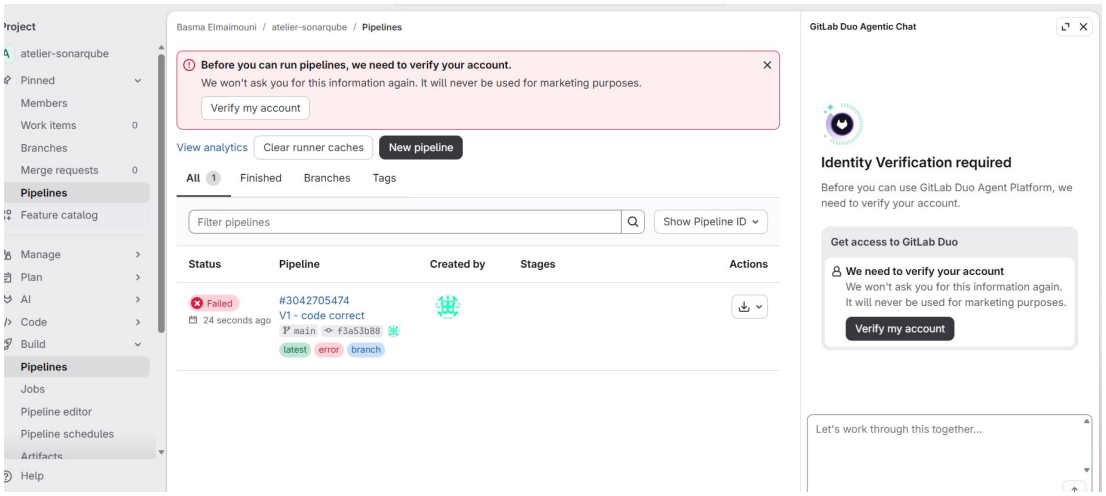


Figure 1 : pipeline #3042705474 en échec, GitLab demande de vérifier le compte.

Après vérification, le job sonarqube-check a échoué à cause de l'Automatic Analysis, encore activée dans SonarQube Cloud :

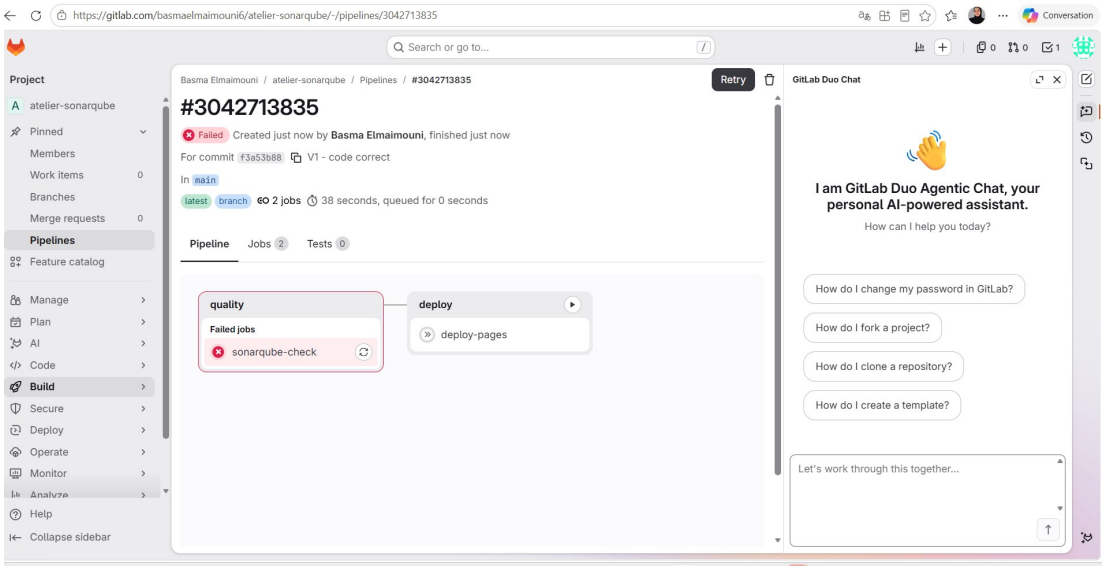


Figure 2 : pipeline #3042713835, sonarqube-check en échec, deploy-pages non exécuté.

```
57  22:31:25  22:31:25.268 INFO  Project key: basmaelmaimouni6_atelier-sonarqube
58  22:31:25  22:31:25.269 INFO  Base dir: /builds/basmaelmaimouni6/atelier-sonarqube
59  22:31:25  22:31:25.272 INFO  Working dir: /builds/basmaelmaimouni6/atelier-sonarqube/.scan
              nerwork
60  22:31:25  22:31:25.301 INFO  Check ALM binding of project 'basmaelmaimouni6_atelier-sonarq
              ube'
61  22:31:25  22:31:25.496 INFO  Detected project binding: BOUND
62  22:31:25  22:31:25.498 INFO  Check ALM binding of project 'basmaelmaimouni6_atelier-sonarq
              ube' (done) | time=194ms
63  22:31:25  22:31:25.504 INFO  Load branch configuration
64  22:31:25  22:31:25.505 INFO  Auto-configuring branch 'main'
65  22:31:25  22:31:25.507 INFO  Load branch configuration (done) | time=3ms
66  22:31:25  22:31:25.512 INFO  Create analysis
67  22:31:25  22:31:25.914 INFO  Branch name: main, type: long
68  22:31:25  22:31:25.916 INFO  Create analysis (done) | time=405ms
69  22:31:26  22:31:26.522 INFO  Load quality profiles
70  22:31:26  22:31:26.894 INFO  Load quality profiles (done) | time=372ms
71  22:31:26  22:31:26.936 INFO  Load active rules
72  22:31:27  22:31:27.676 INFO  Load active rules (done) | time=737ms
73  22:31:28  22:31:28.321 ERROR You are running CI analysis while Automatic Analysis is enabl
              ed. Please consider disabling one or the other.
74  22:31:28  22:31:28.345 INFO  EXECUTION FAILURE
75  22:31:28  22:31:28.346 INFO  Total time: 15.616s
76  22:31:29  Cleaning up project directory and file based variables          00:00
77  22:31:29  ERROR: Job failed: exit code 3
```

Figure 3 : log du job : « You are running CI analysis while Automatic Analysis is enabled ».

La désactivation de l'Automatic Analysis (Administration > Analysis method) puis un Retry ont résolu le problème.

4. Version 1 : code correct

4.1 Pipeline réussi

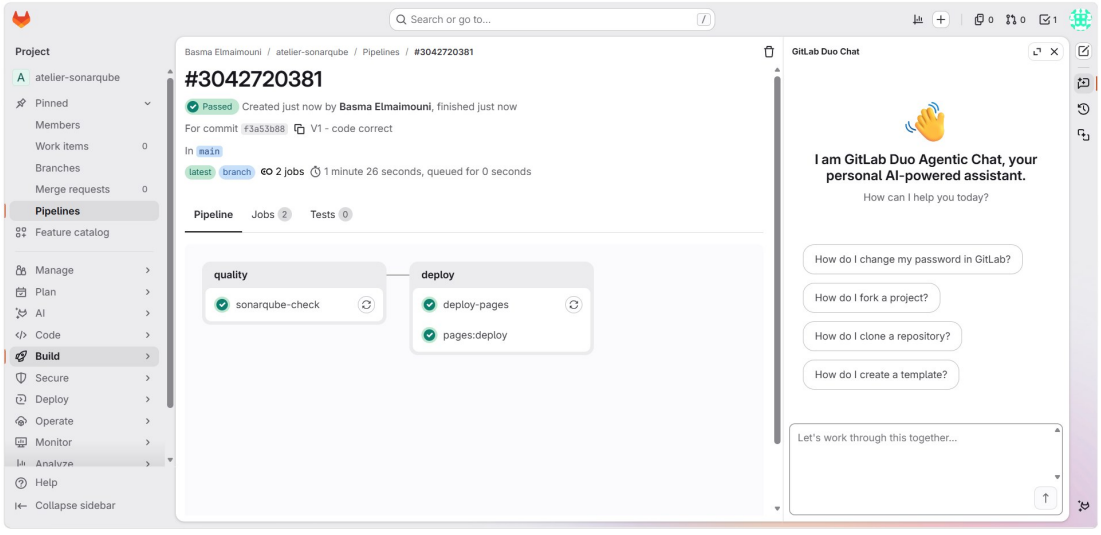


Figure 4 : pipeline V1, jobs sonarqube-check et deploy-pages réussis.

Les deux jobs réussissent : le code est analysé par SonarQube puis déployé. Comme `sonar.qualitygate.wait=true` est activé, la réussite de sonarqube-check confirme que le Quality Gate est respecté.

4.2 Site déployé sur GitLab Pages



Figure 5 : site publié, après un clic sur le bouton le JavaScript fonctionne.

5. Version 2 : code vulnérable

Le code est modifié pour lire un paramètre de l'URL et l'insérer dans la page avec innerHTML :

```
const params = new URLSearchParams(window.location.search);
const message = params.get("message");

document.getElementById("message").innerHTML = message;
```

Le fichier .gitlab-ci.yml est enrichi avec trois stages : quality, security et deploy (jobs syntax-check, sonarqube-check, security-check et deploy-pages).

Remarque : une première variante, où innerHTML recevait la valeur d'un champ de formulaire, n'a pas été détectée par SonarQube (pipeline réussi, aucune issue). L'analyseur n'y voyait pas de source de données non fiable. Avec window.location.search, une source contrôlable par un attaquant, la faille est bien détectée.

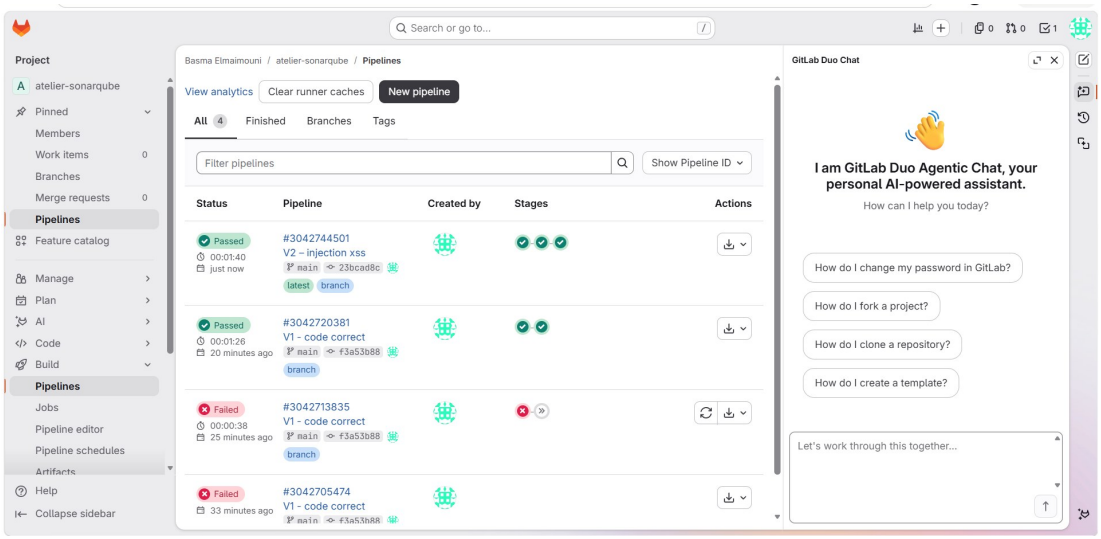


Figure 6 : première variante (champ de formulaire) : pipeline #3042744501 réussi.

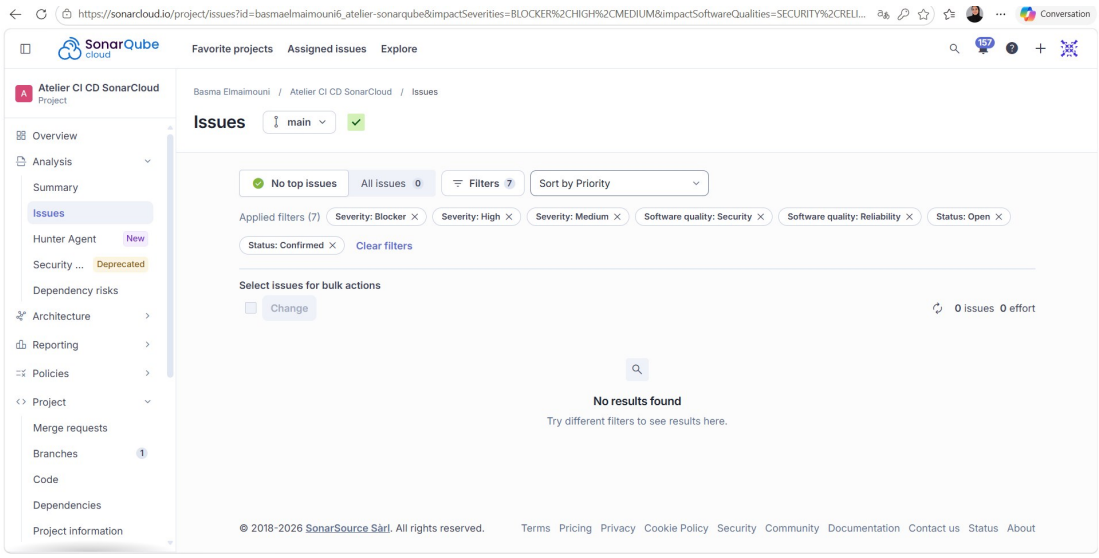


Figure 7 : première variante : SonarQube Cloud ne détecte aucune issue.

5.1 Dépôt GitLab après le push

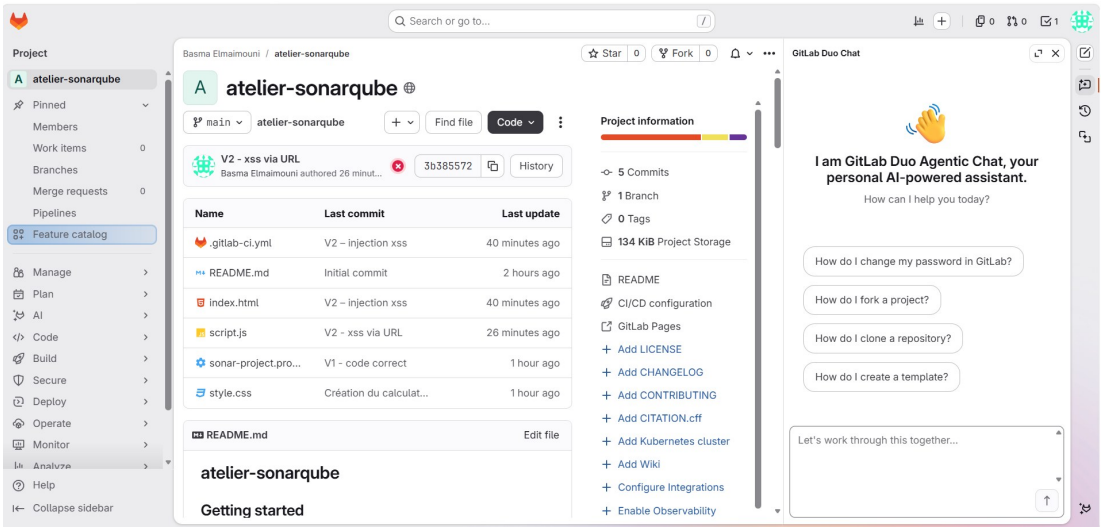


Figure 8 : commit V2 en échec (croix rouge) avec les fichiers du projet.

5.2 Pipeline échoué

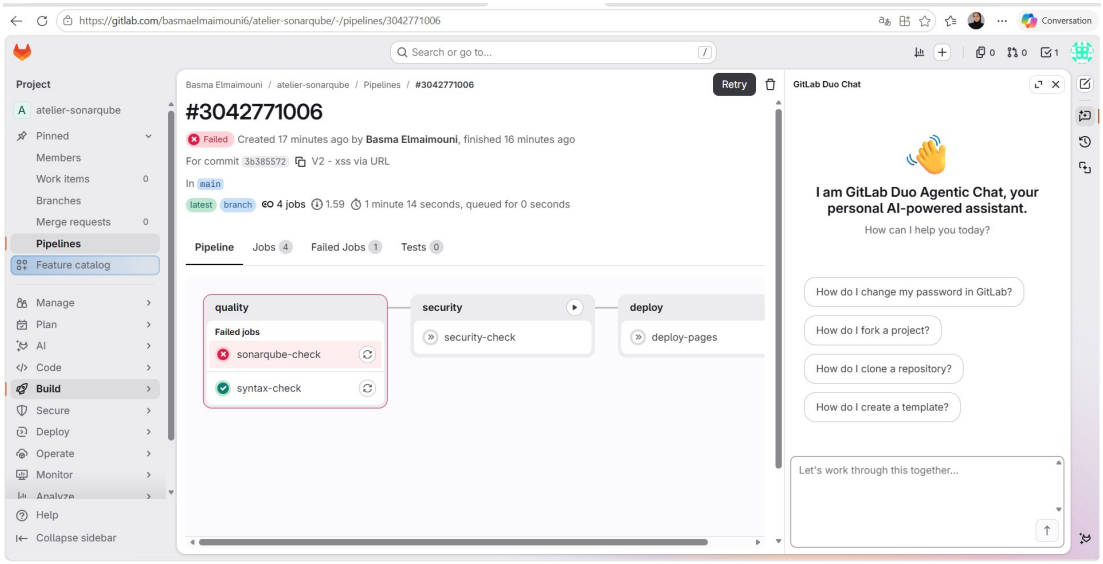


Figure 9 : pipeline V2, sonarqube-check en échec, security-check et deploy-pages non exécutés.

Le job sonarqube-check échoue, donc les stages suivants ne s'exécutent pas : le déploiement est bloqué.

5.3 Résultat SonarQube

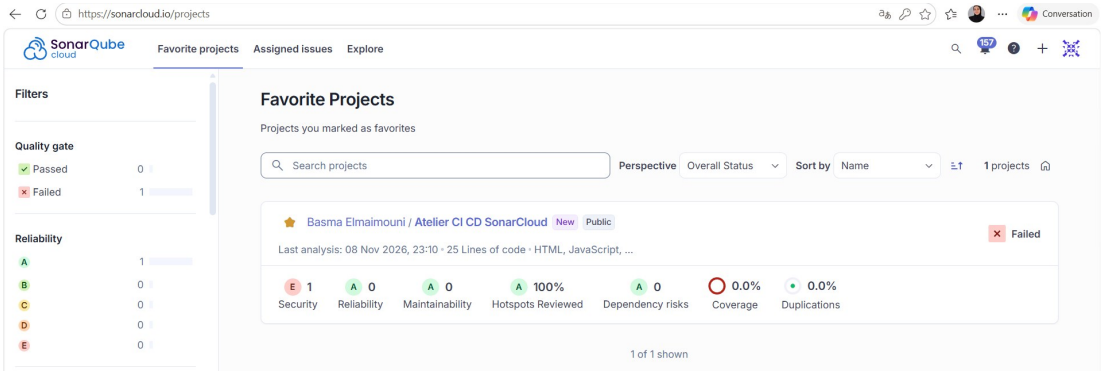


Figure 10 : Quality Gate « Failed », Security E avec 1 problème.

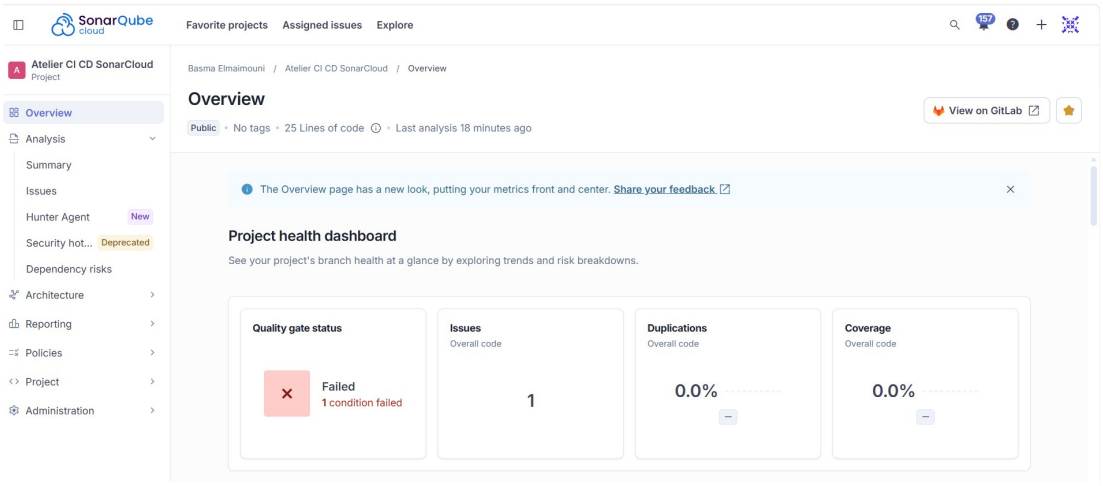


Figure 11 : vue d'ensemble du projet : Quality Gate Failed (1 condition failed).

5.4 Détail de la vulnérabilité

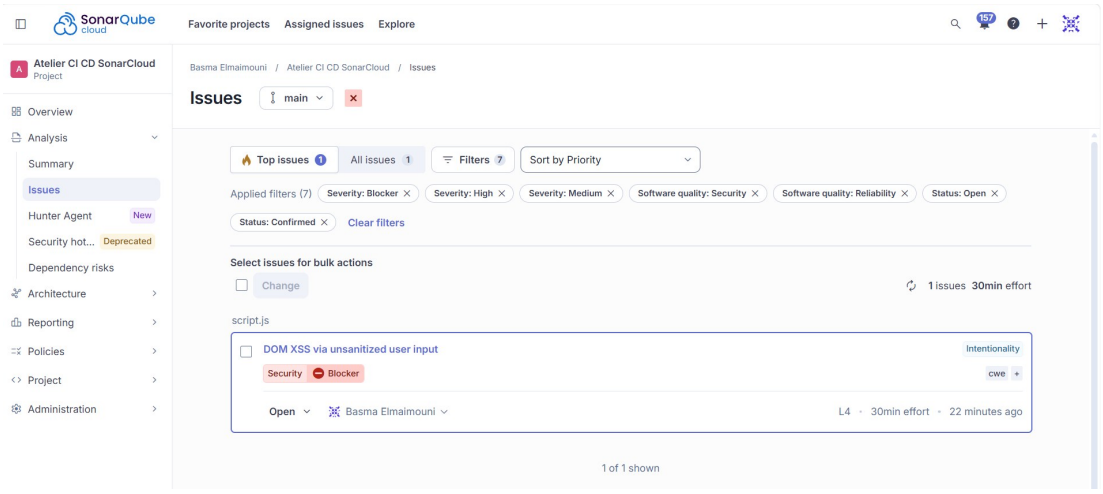


Figure 12 : DOM XSS via unsanitized user input, Blocker, script.js ligne 4.

La donnée provenant de l'URL est insérée dans le DOM via innerHTML sans nettoyage : un attaquant peut injecter du code JavaScript.

6. Correction proposée

Remplacer innerHTML par textContent, qui traite la saisie comme du texte et non comme du HTML :

```
document.getElementById("message").textContent = message;
```

Après ce correctif et un nouveau git push, le pipeline réussit et le Quality Gate repasse à « Passed ».

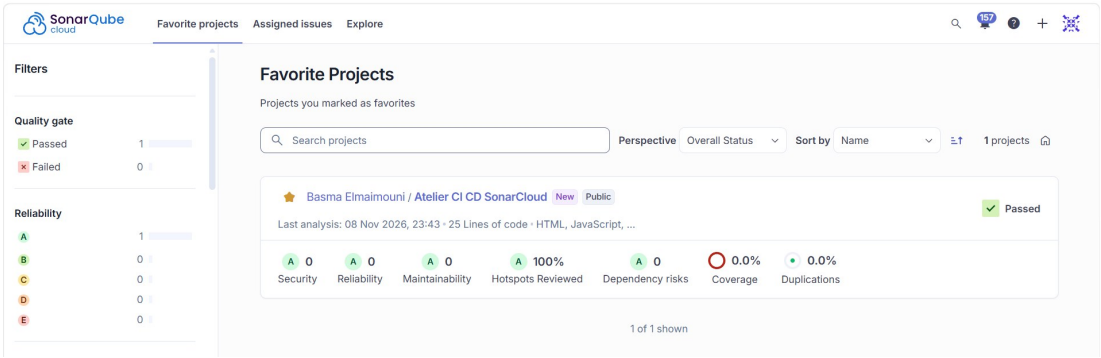


Figure 13 : Quality Gate « Passed » après correction (Security A, 0 problème).

Le formulaire de démonstration affiche ensuite la saisie comme du simple texte :



Figure 14 : saisie du texte « salut » dans le formulaire.



Figure 15 : après clic sur Afficher, le texte saisi est affiché dans la page.

7. Conclusion

1. Le développeur pousse son code sur GitLab.
2. GitLab déclenche automatiquement le pipeline CI/CD.
3. SonarQube Cloud analyse le code et détecte la faille XSS.
4. Le Quality Gate échoue et le job sonarqube-check échoue.
5. Le déploiement est bloqué : le code vulnérable n'atteint jamais le site.
6. Le développeur corrige, pousse à nouveau, et le cycle recommence.

À retenir : GitLab CI/CD automatise les étapes, SonarQube contrôle la qualité et la sécurité du code, et le Quality Gate empêche le déploiement d'un code qui ne respecte pas les critères. SonarQube ne remplace ni les tests, ni la revue de code.